CASPER

CS683: Advanced Computer Architecture

Lecture 14: Cache Coherence & Memory Consistency

https://www.cse.iitb.ac.in/~biswa/courses/

https://www.cse.iitb.ac.in/~biswa/

Cache Coherence Performance

Coherence misses: misses due to accesses to blocks that were invalidated due to coherence events

True sharing: misses that occur when multiple threads share the same data item (byte/word)

False sharing: misses that occur when multiple threads share different data items located in a single cache block

Miss Latency in (cycles/ns) With coherence states for core-0

E State

Source	L1 hit	L2 hit	L3 hit	DRAM hit
Local	4	10	38	65
Core 1 - 3	65	65	**65**	65
Core 4 - 7	186	186	**186**	106

M State

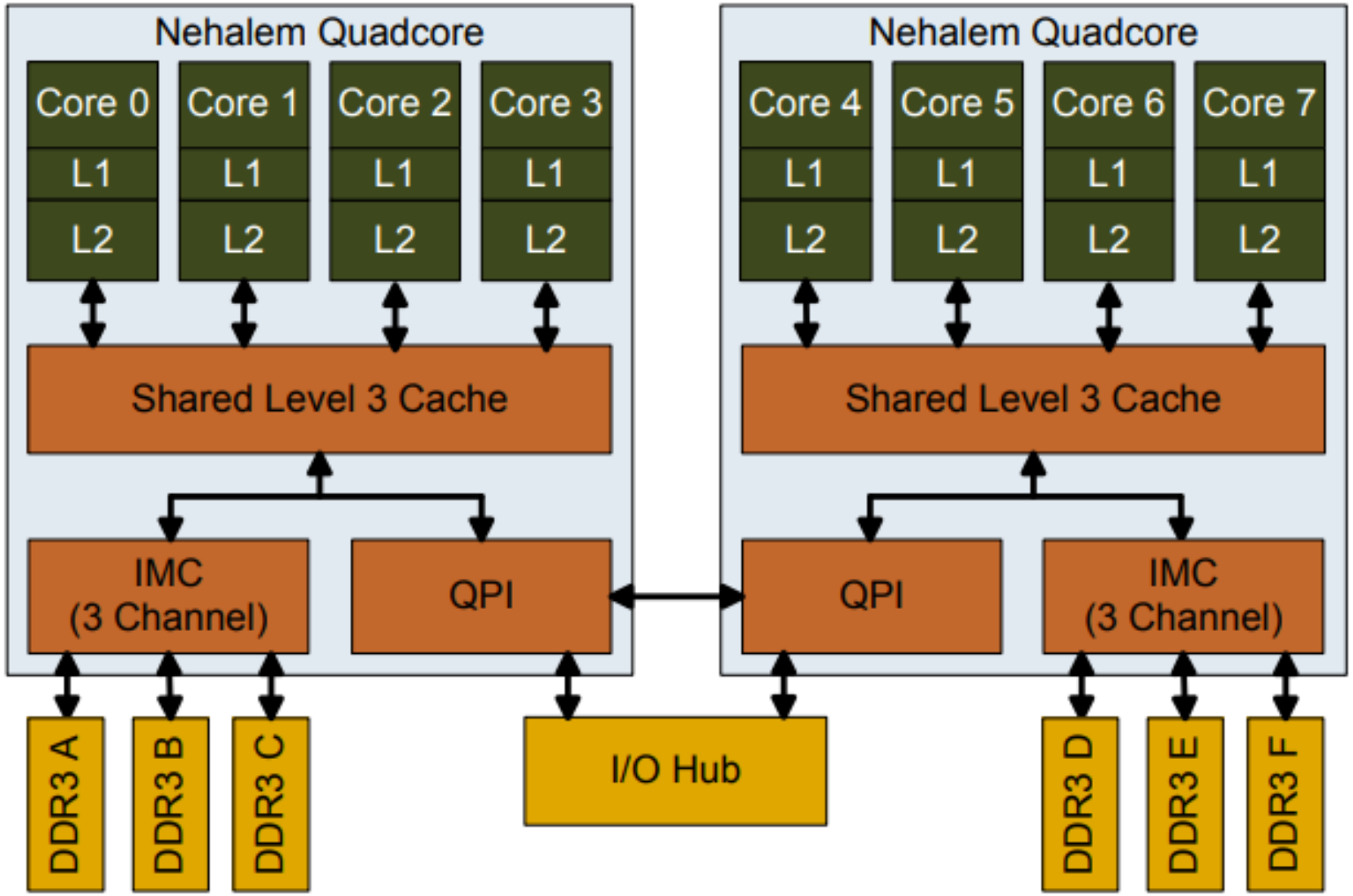
Source	L1	L2	L3	DRAM
Local	4	10	38	65
Core 1 - 3	-	-	**38**	65
Core 4 - 7	109	109	**109**	109

Miss Latency for Core 0

S State

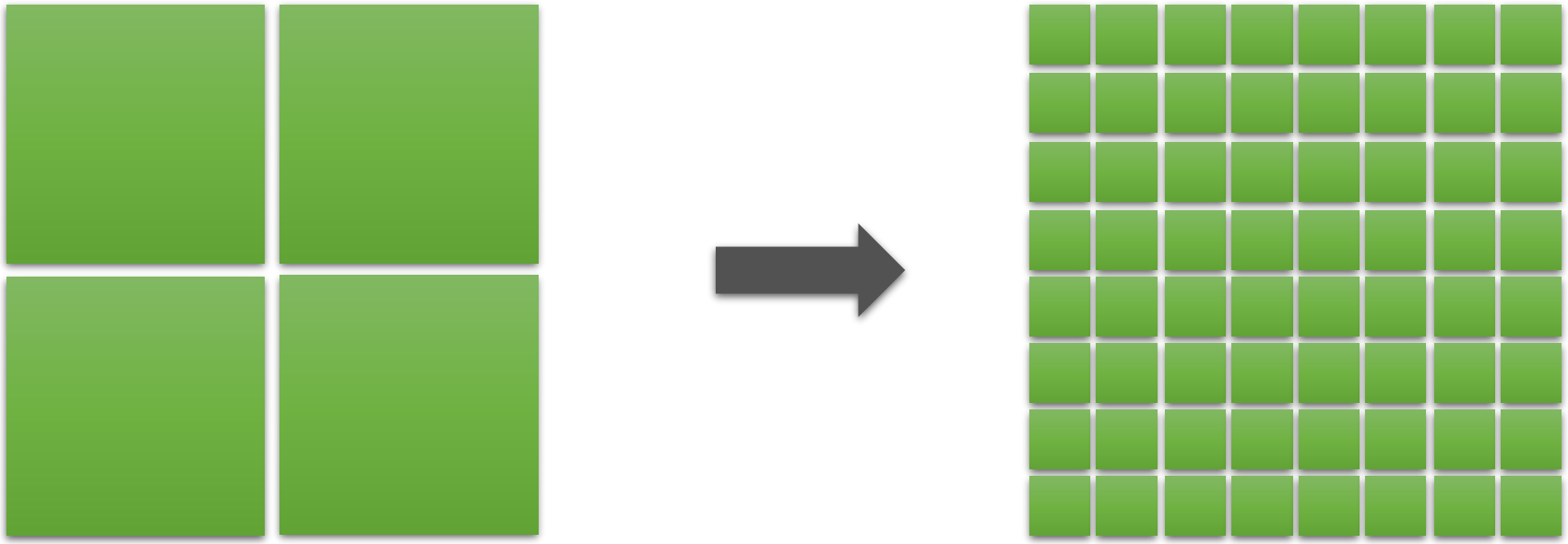
Source	L1	L2	L3	DRAM
Local	4	10	38	65
Core 1 - 3	38	38	38	65
Core 4 - 7	170	170	170	106

The latencies to local caches are independent of the coherency state since the data can be read directly in all states.



Adapted from PACT 2009

Multicore to Manycore (Think about coherence performance)

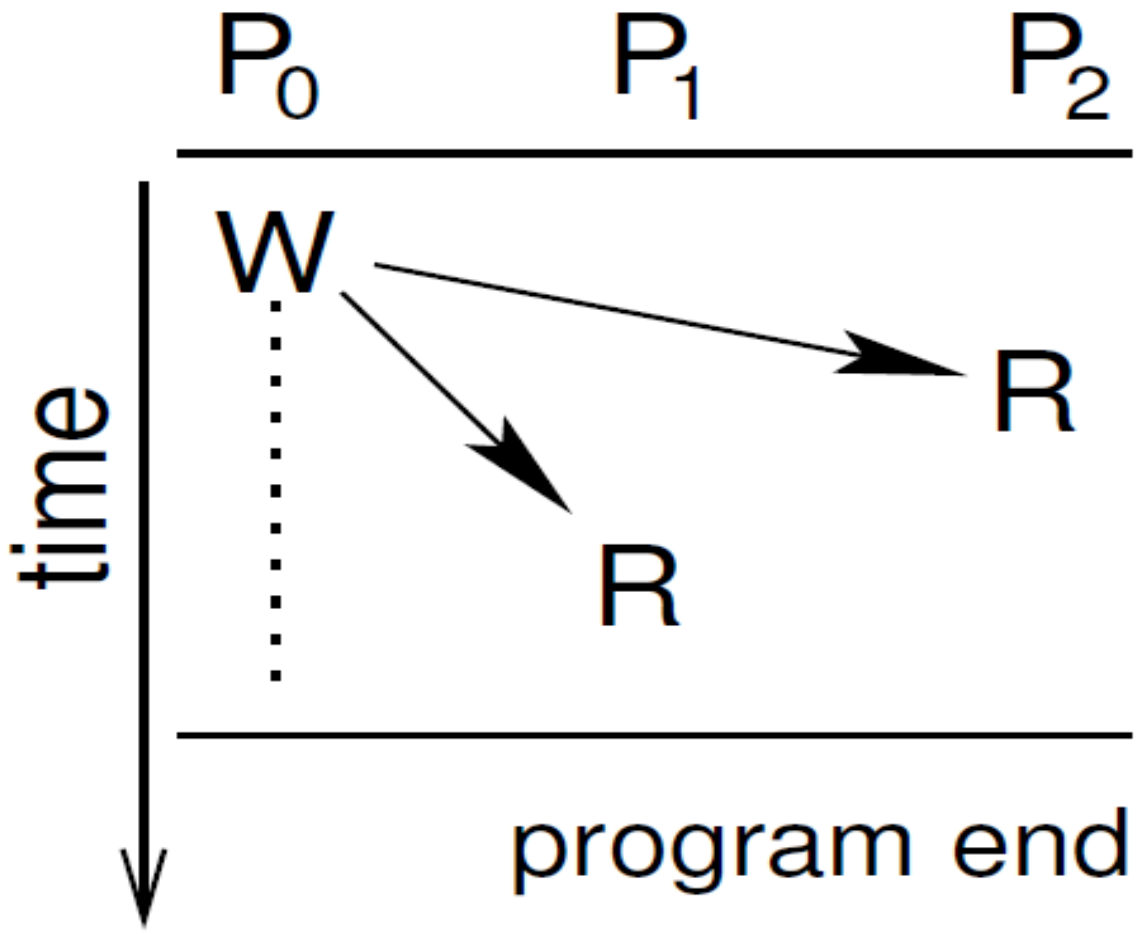


Coherence and Sharing Patterns

Definition - A cache line is described as shared if it is written to or read from by more than one processor/core.

The number of cores involved and the ordering of reads and writes indicates a certain sharing pattern.

Read Only Pattern



Read Only Pattern - 2

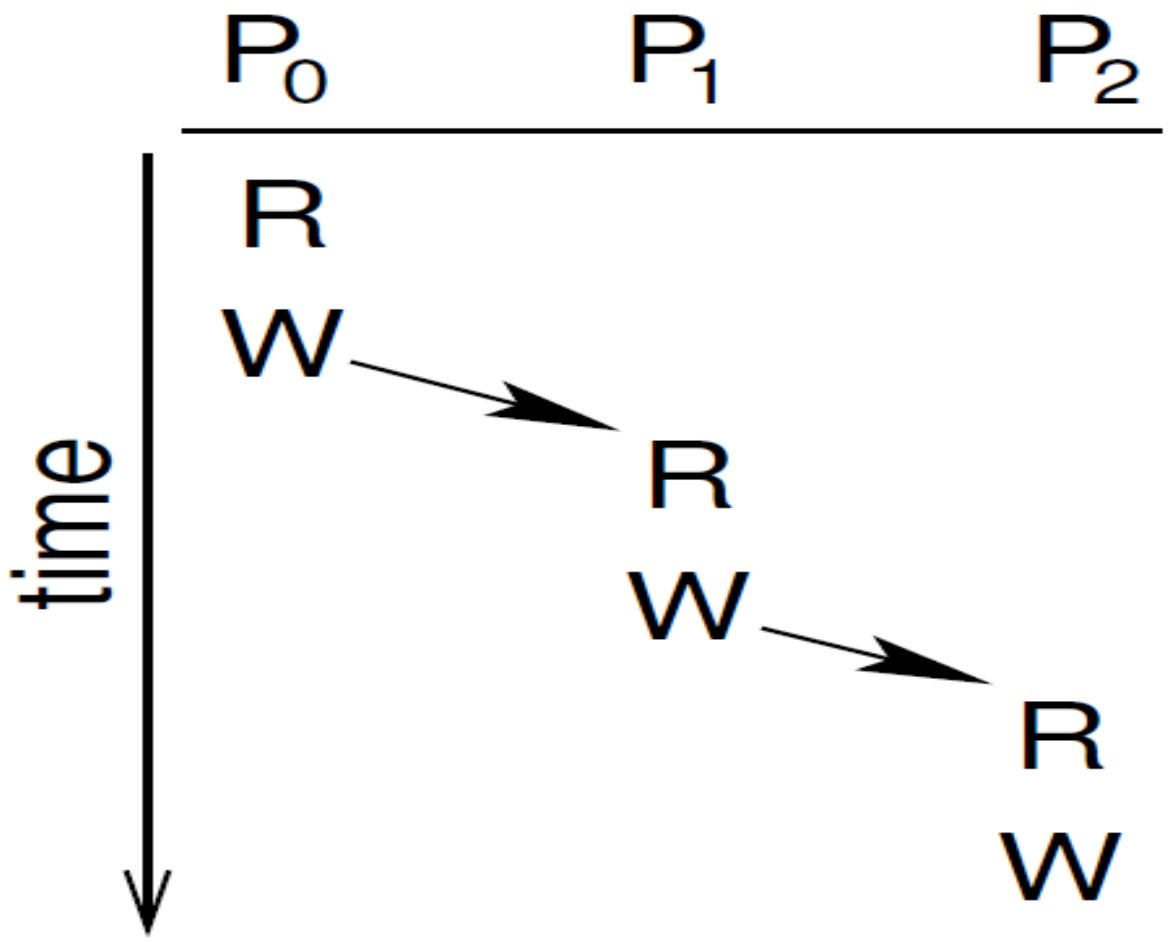
- Dominant Coherence States or Frequent Coherence state transitions ??
- Coherence/Communication Traffic - ??

Coherence Traffic - Traffic generated because of transfer of coherence messages.

- Examples - ??

Input File is read in the sequential phase, and then used by all the threads in the parallel phase.

Migratory Pattern

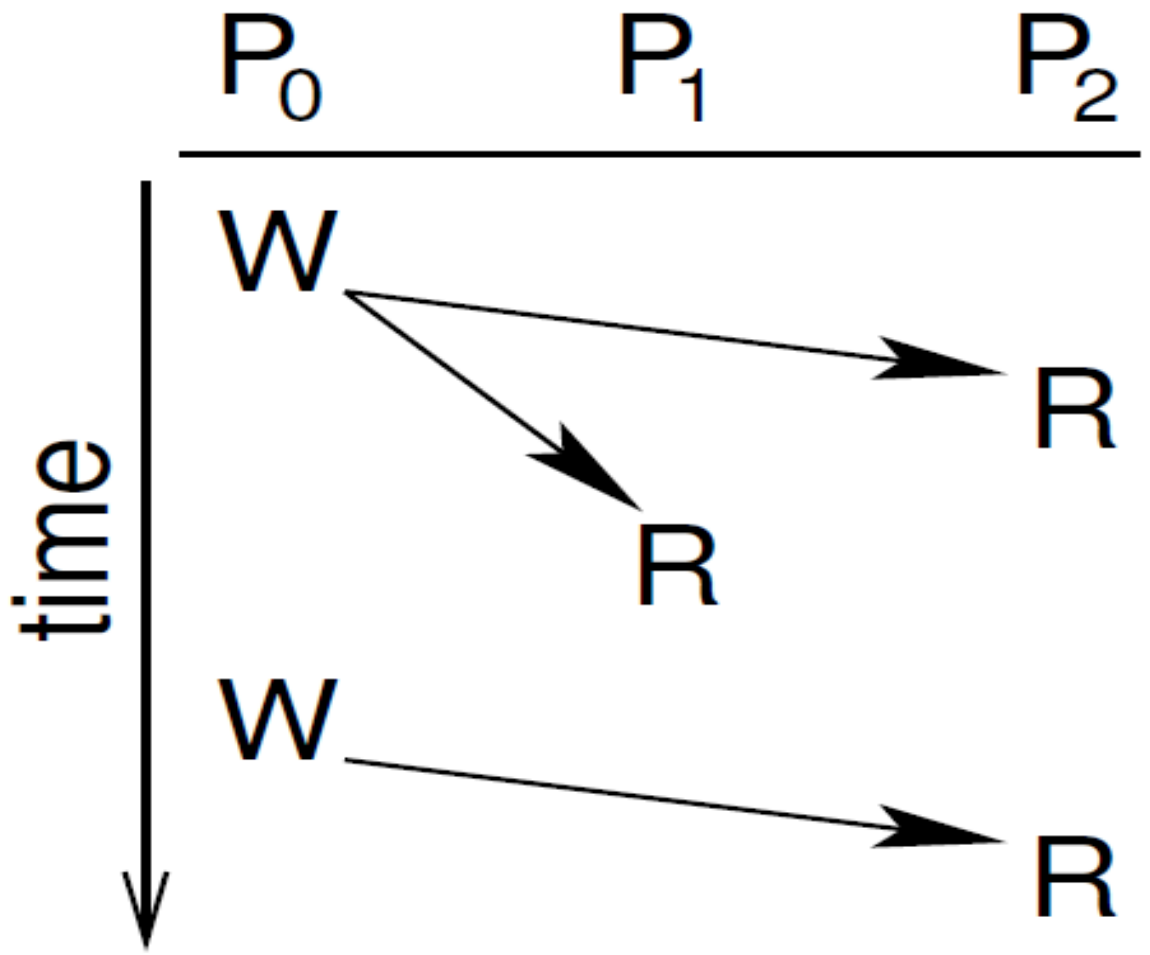


Migratory Pattern-2

- Frequent Coherence State transitions ??
- Communication Traffic ??
- Examples ??

Code inside Critical/atomic Regions.

Producer-Consumer Pattern



Producer-Consumer - 2

- Frequent Coherence State Transitions ??

- Communication Traffic ??

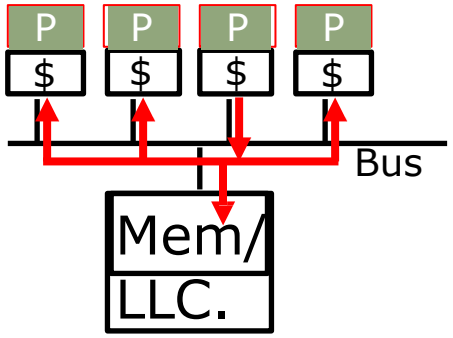
  Ping-Pong Behavior

- Examples ??

Readers-Writers Problem

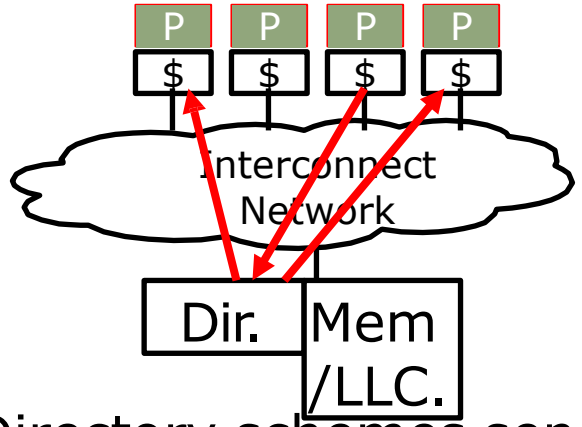
Too much traffic: Cache Coherence Directory is the solution

Snoopy Protocols (Goodman 1983)



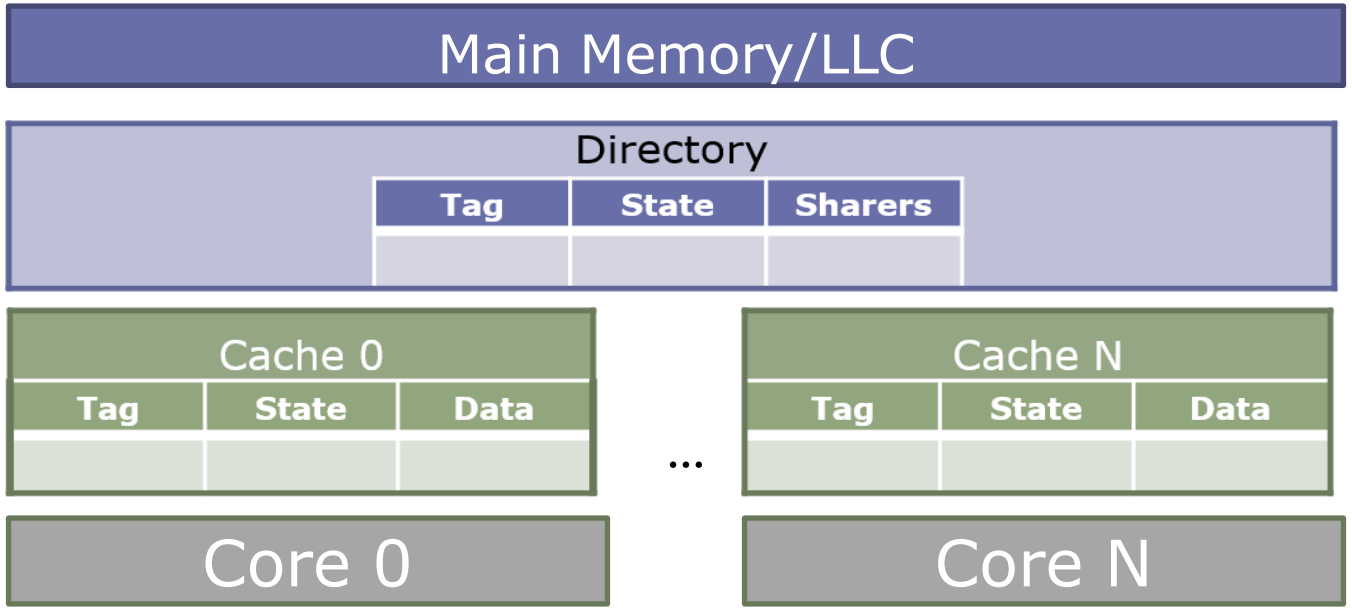
- Snoopy schemes *broadcast* requests over memory bus
- Difficult to scale to large numbers of processors
- Requires additional bandwidth to cache tags for snoop requests

Directory Protocols



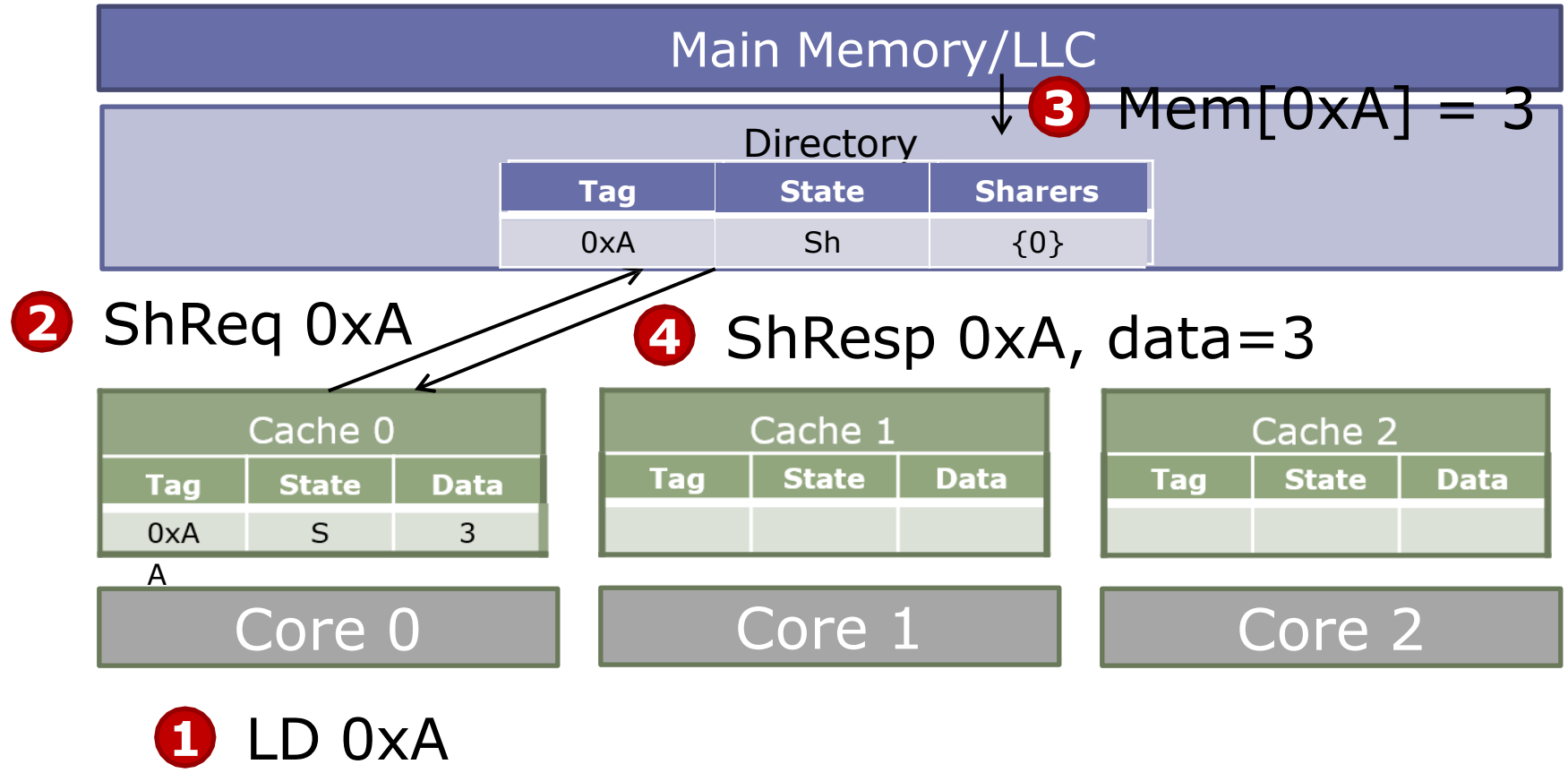
- Directory schemes send messages to *only those caches that might have the line*
- Can scale to large numbers of processors
- Requires extra directory storage to track possible sharers

MSI Directory Protocol

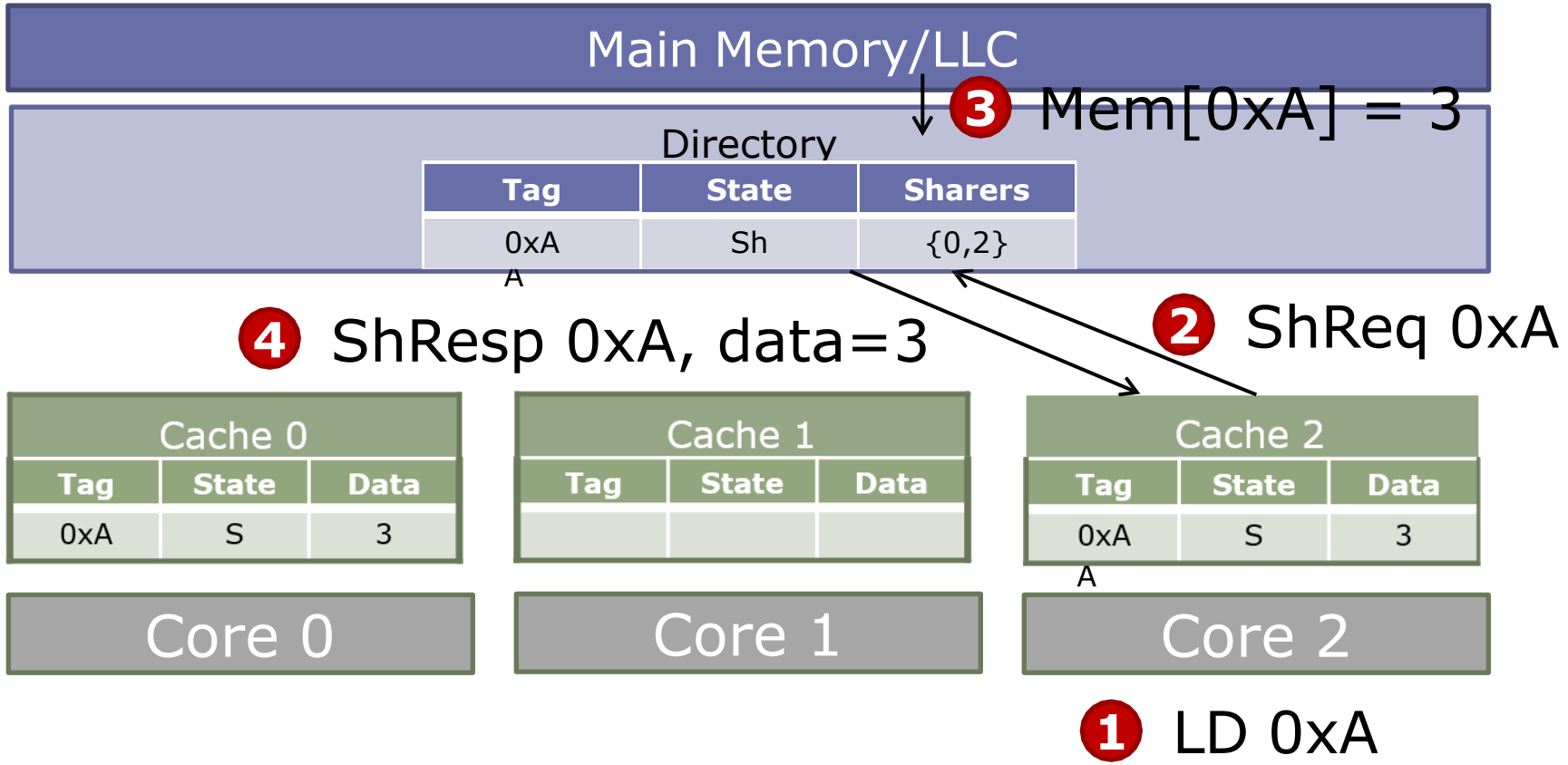


- Cache states: Modified (M) / Shared (S) / Invalid (I)
- Directory states:
  - Uncached (Un): No sharers
  - Shared (Sh): One or more sharers with read permission (S)
  - Exclusive (Ex): A single sharer with read & write permissions (M)
- Transient states not drawn for clarity; for now, assume no racing requests

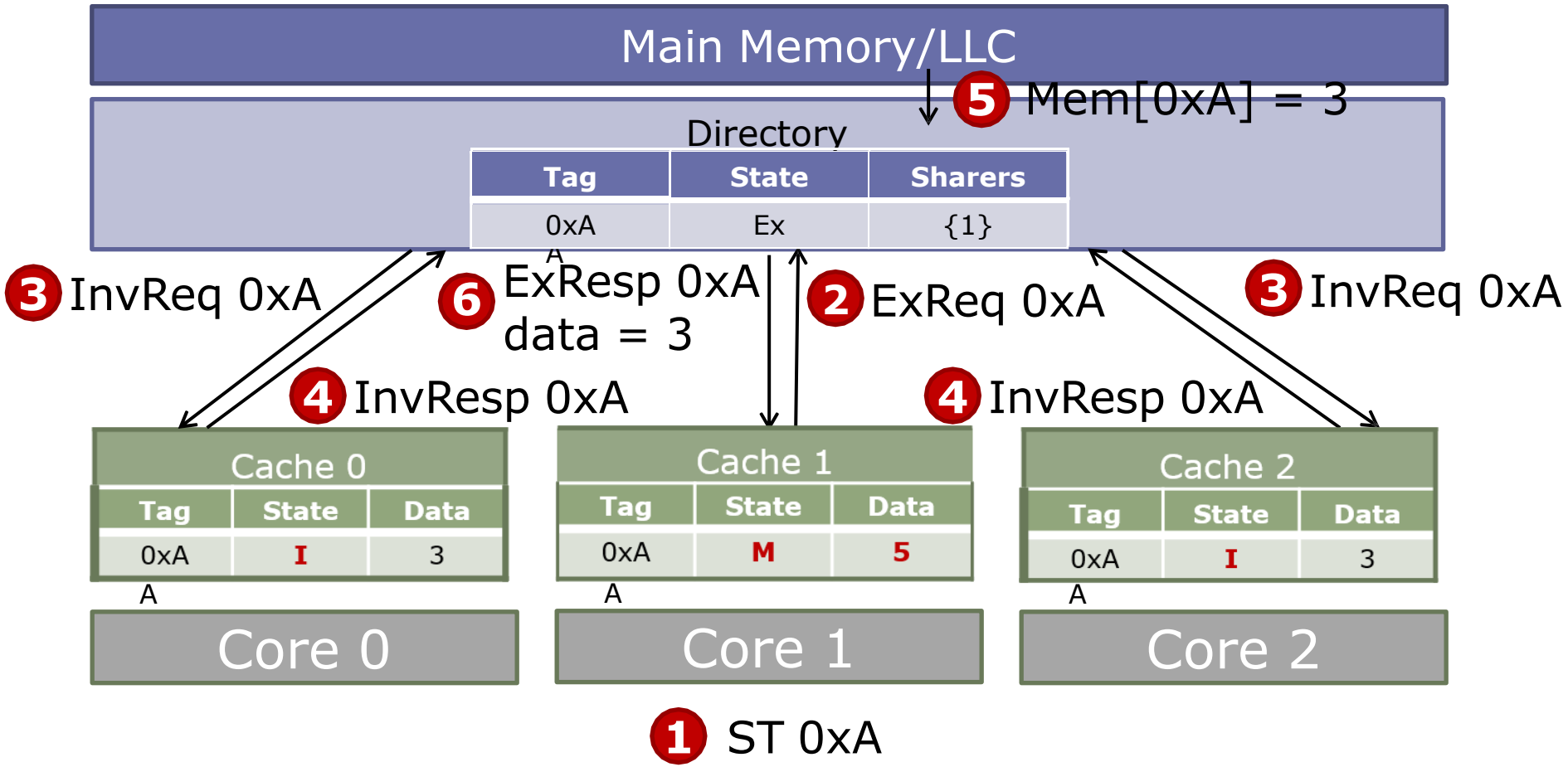
Example


Main Memory/LLC
3 Mem[0xA] = 3
Directory
Tag
State
Sharers
0xA
Sh
{0}
2 ShReq 0xA
4 ShResp 0xA, data=3
Cache 0
Tag
State
Data
0xA
S
3
A
Cache 1
Tag
State
Data
Cache 2
Tag
State
Data
Core 0
Core 1
Core 2
1 LD 0xA

Example-Contd



Example-Contd.



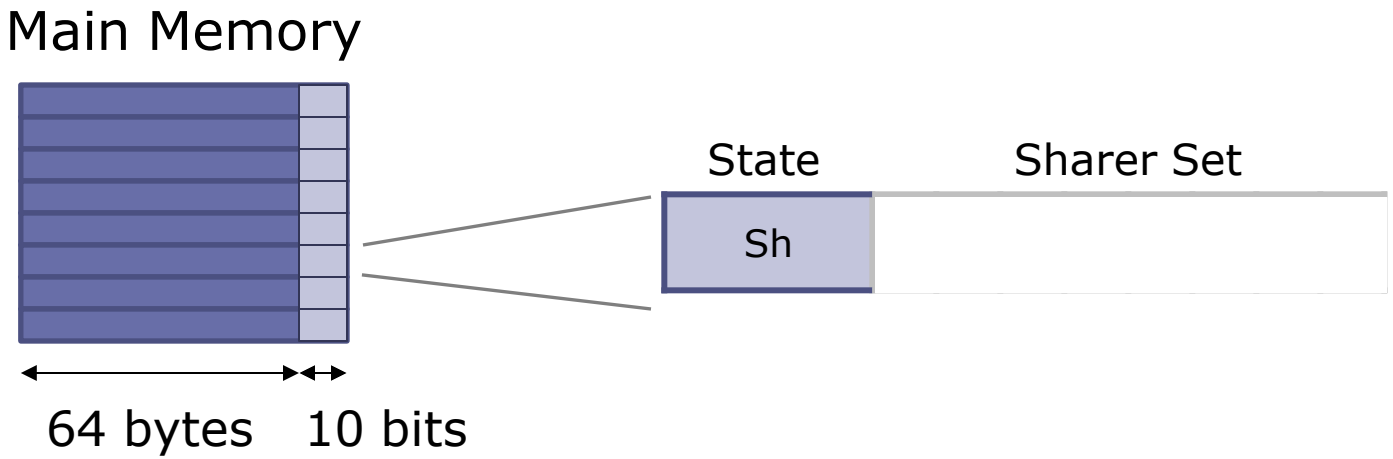
Directory Organization

- Requirement: Directory needs to keep track of all the cores that are sharing a cache block

- Challenge: For each block, the space needed to hold the list of sharers grows with number of possible sharers…

Flat Directory

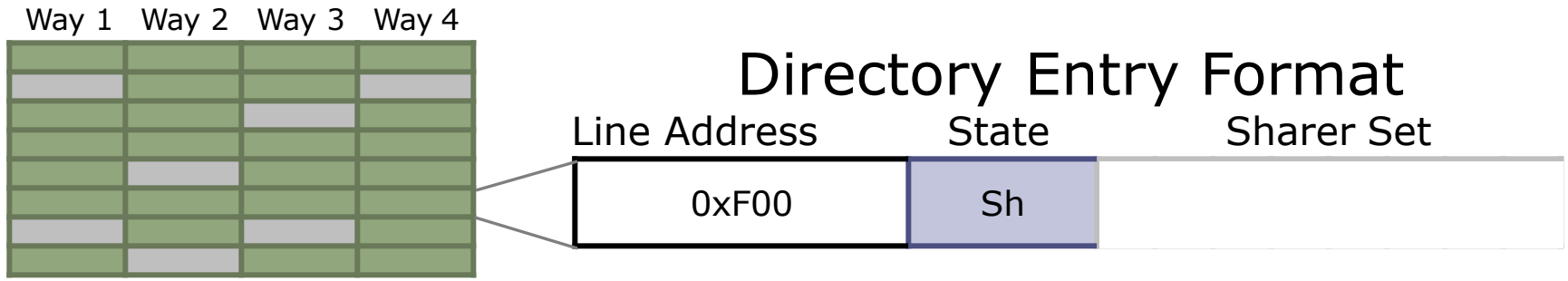
- Dedicate a few bits of main memory to store the state and sharers of every line
- Encode sharers using a bit-vector



✓ Simple
✗ Slow
✗ Very inefficient with many processors (~P bits/line)

Sparse Directory

- Not every line in the system needs to be tracked – only those in private caches!
- Idea: Organize directory as a cache



✓ Low latency, energy-efficient

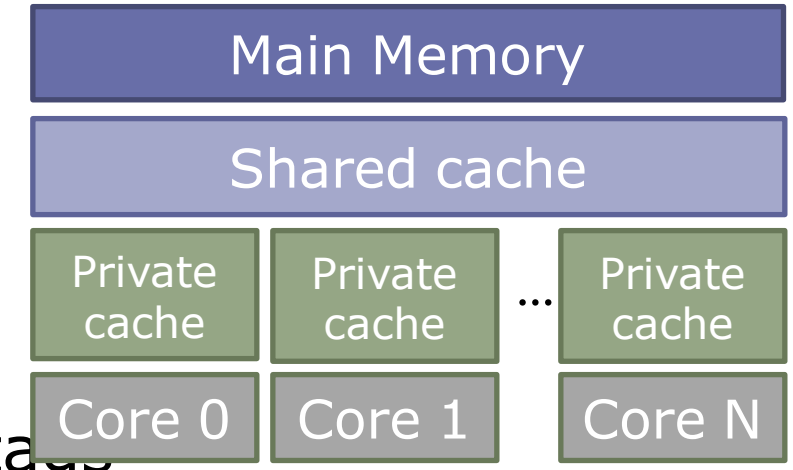
✗ Bit-vectors grow with # cores → Area scales poorly

Some more enhancements

- Coarse-grain bit-vectors (e.g., 1 bit per 4 cores)
  - Sharer Set
    - `0-3  4-7  8-11  12-15  16-19  20-23`

Coherence Directory in 2024

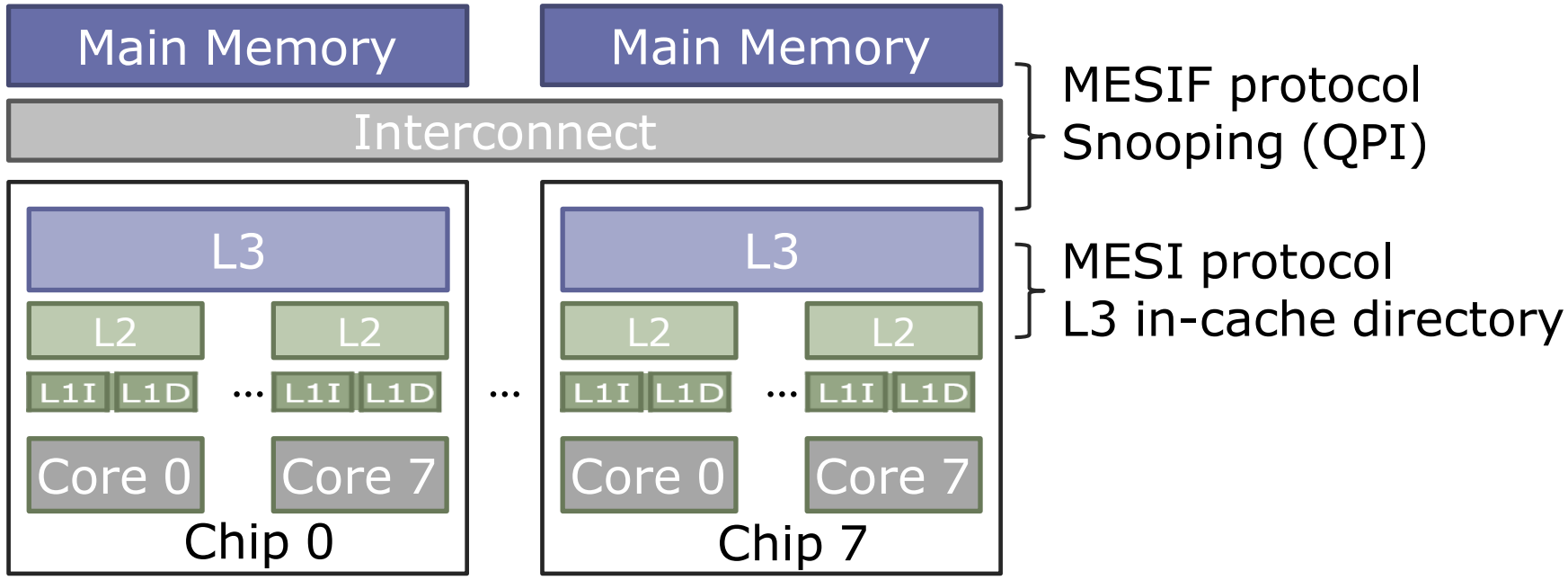
- Common multicore memory hierarchy:
  - 1+ levels of private caches
  - A shared last-level cache
  - Need to enforce coherence among private caches



- Idea: Embed the directory information in shared cache tags
  - Shared cache must be inclusive (life is easy)
  - Need extended directory if non-inclusive (Intel Skylake-X/SP)

Multi-socket Coherence

- Can use the same or different protocols to keep coherence across multiple levels
- Key invariant: Ensure sufficient permissions in all intermediate levels
- Example: 8-socket Xeon E7 (8 cores/socket)



Remember Inclusive Cache ☺

- Inclusion traffic on top of coherence traffic as back-invalidations will go to each core's private caches.

- Non-inclusive caches with extended directory is a win-win tradeoff with additional design complexity.

From Coherence to Consistency

- Cache coherence makes private caches invisible to software
  - Concerns reads/writes to a single memory location

- Memory consistency models precisely specify how memory behaves with respect to read and write operations from multiple processors (caches do no matter anymore)
  - Concerns reads/writes to multiple memory locations

Memory Consistency

- The memory consistency model of a shared-memory system determines the order in which memory operations will appear to execute to the programmer.
  - Core 1 writes to some memory location…
  - Core 2 reads from that location…
  - Do we get the result we expect?

Coherence vs Consistency

Formally:

– Coherence: What values can a read return?

• Concerns reads/writes to a single memory location

– Consistency: When do writes become visible to reads?

• Concerns reads/writes to multiple memory locations

Why it matters?

Initial memory contents

a: 0 // memory location a storing zero

flag: 0 // memory location flag storing zero

Processor 1		*Processor 2*
Store (a), 10;	L:	Load r1, (flag);
Store (flag), 1;		if r_1 == 0 goto L;
		Load r2, (a);

- What value does r2 hold after both processors finish running this code?

Why it matters?

Initial memory contents

a: 0 // memory location a storing zero

flag: 0 // memory location flag storing zero

Processor 1	*Processor 2*
Store (a), 10;	L: Load r1, (flag);
Store (flag), 1;	if r_1 == 0 goto L;
	Load r2, (a);

- What value does r2 hold after both processors finish running this code?

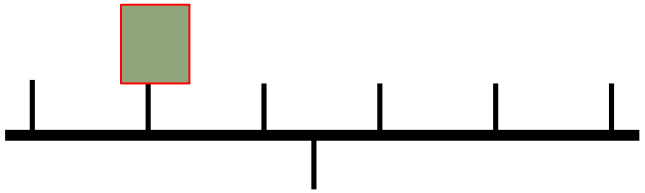
It depends on the order in which processor 2 observes processor 1's stores!

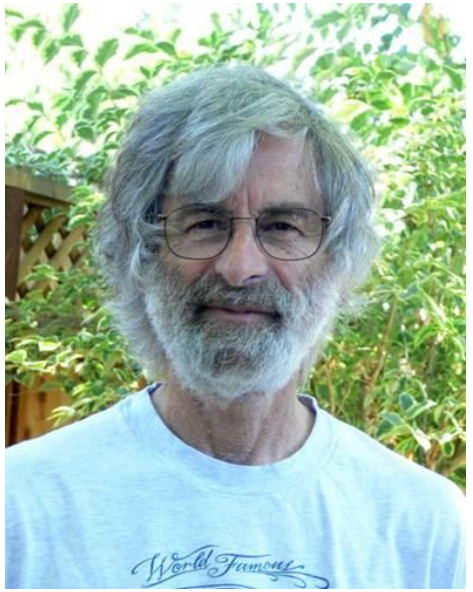
10 if Store (flag) > Store (a); 0 or 10 otherwise

The definition, An important one

memory consistency model, or, more simply, a *memory model*, is a specification of the allowed behavior of multithreaded programs executing with shared memory. For a multithreaded program executing with specific input data, it specifies what values dynamic loads may return. Unlike a single-threaded execution, multiple correct behaviors are usually allowed.

The definition



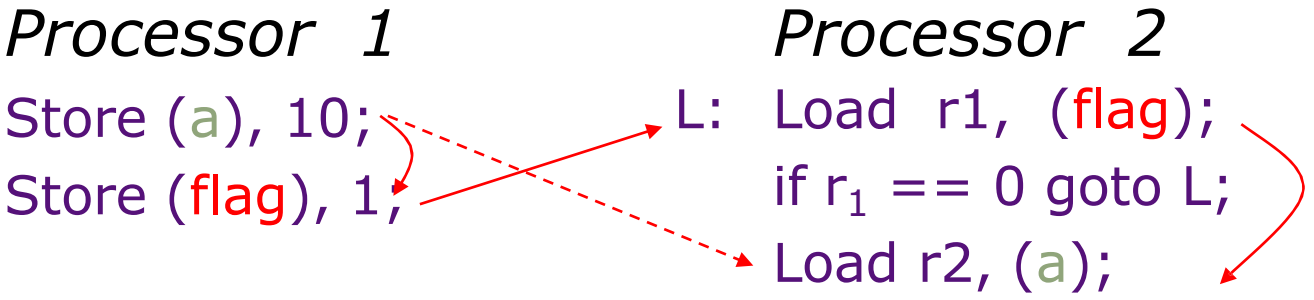


"A system is *sequentially consistent* if the result of any execution is the same as if the operations of all the processors were executed in some sequential order, and the operations of each individual processor appear in the order specified by the program"

Leslie Lamport, Turing award winner

Sequential Consistency =
arbitrary *order-preserving interleaving*
of memory references of sequential programs

Let's continue



- In-order instruction execution
- Atomic loads and stores

SC is easy to understand, but architects and compiler writers want to violate it for performance

Trend so far ☹

Architectural optimizations that are correct for uniprocessors often violate sequential consistency and result in a new memory model for multiprocessors

Memory Consistency Models

- Sequential Consistency
  - All reads and writes in order

- Relaxed Consistency (one or more of the following)
  - Loads may be reordered after loads
    - e.g., PA-RISC, Power, Alpha
  - Loads may be reordered after stores
    - e.g., PA-RISC, Power, Alpha
  - Stores may be reordered after stores
    - e.g., PA-RISC, Power, Alpha, PSO
  - Stores may be reordered after loads
    - e.g., PA-RISC, Power, Alpha, PSO, TSO, x86

  - Other more esoteric characteristics
    - e.g., Alpha

X86 uses TSO (Total Store Order)

- We will come back in two weeks time. Once we understand how the processor looks at stores.

- So far we have seen how caches see stores ☺